

Consegna 7a-CUDA (rel da consegnare)

Esercizi di Base CUDA

1. `cuda1_printf.cu` (gerarchia e asincronia)

Lancia una griglia di 4 blocchi, ognuno con 8 thread. Stampa gli ID di thread e blocco.

```
[mariaandrea.zoccoli@ui01 base]$ cat cuda1_printf.out
module
#SLURM_JOB_NODELIST      : wn41
#CUDA_VISIBLE_DEVICES    : 0
```

Ogni thread stampa le coordinate del blocco nella griglia e le proprie coordinate all'interno del blocco.

```
Hello from block 1/4, thread 0/8
[...]
Hello from block 0/4, thread 7/8
```

OSS: l'ordine di esecuzione dei blocchi non è sequenziale. Lo scheduler hardware della GPU distribuisce i blocchi ai vari SM in modo asincrono e indipendente, non garantendo un ordine predefinito

2. `cuda2_sumArray.cu`

Inizializza due array da 10 elementi sulla RAM della CPU (Host), prenota spazio sulla VRAM della GPU (device), copia i dati tramite bus PCIe, fa sommare ogni coppia di elementi da un thread distinto e riporta il risultato sulla CPU.

```
[mariaandrea.zoccoli@ui01 base]$ cat cuda2_sumArray.out
[...]
Host> A: 1.0 2.0 3.0 4.0 5.0 6.0 7.0 8.0 9.0 10.0
Host> B: 1.0 2.0 3.0 4.0 5.0 6.0 7.0 8.0 9.0 10.0
sommaarray<<<DimGrid,DimBlock>>>
Host> C: 2.0 4.0 6.0 8.0 10.0 12.0 14.0 16.0 18.0 20.0
```

OSS: problema memory-bound. L'overhead delle latenze di trasferimento dati supera il tempo di elaborazione effettiva della GPU.

3. `cuda3_matMul.cu`

Moltiplica due matrici 3×3 . Invece di usare un vettore lineare, mappa i thread su una griglia bidimensionale usando le coordinate `.x` e `.y`.

```
[mariaandrea.zoccoli@ui01 base]$ cat cuda3_matMul.out
[...]
Host> A:
-1.0 0.0 1.0
2.0 3.0 4.0
5.0 6.0 7.0
```

```
Host> B:
1.0 2.0 3.0
4.0 5.0 6.0
7.0 8.0 9.0
matMul<<<DimGrid,DimBlock>>>
Host> C:
6.0 6.0 6.0
42.0 51.0 60.0
78.0 96.0 114.0
```

4. `cuda4_riduzione_blocco.cu`

Somma gli elementi di un array sfruttando la memoria Shared usando la tecnica della riduzione ad albero logaritmico.

```
[mariaandrea.zoccoli@ui01 base]$ cat cuda4_riduzione_blocco.out
[...]
c=64.0
```

OSS: l'utilizzo della shared memory permette di risolvere la riduzione limitando le letture e scritture verso la memoria globale, limitandole al solo risultato finale.

5. `cuda5_constant.cu` e `cuda5_managed.cu`

Test sulle memorie constant e unified.

```
[mariaandrea.zoccoli@ui01 base]$ cat cuda5_constant.out
[...]
5
[...]
5
```

```
[mariaandrea.zoccoli@ui01 base]$ cat cuda5_managed.out
[...]
0: Vett*Vett = 0
1: Vett*Vett = 1
2: Vett*Vett = 4
[...]
30: Vett*Vett = 900
31: Vett*Vett = 961
```

6. `cuda6_riduzione_griglia.cu`

Estensione della riduzione sull'intera griglia tramite un meccanismo di sincronizzazione globale per sommare i risultati parziali dei singoli blocchi.

```
[mariaandrea.zoccoli@ui01 base]$ cat cuda6_riduzione_griglia.out
[...]

32.00 32.00 32.00 32.00 32.00 32.00 32.00 32.00 32.00 32.00 32.00 32.00
32.00 32.00 32.00 32.00 32.00 32.00 32.00 32.00 32.00 32.00 32.00 32.00
```

```
32.00 32.00 32.00 32.00 32.00 32.00 32.00 32.00 Block 27 thread 0 entering c
ritical section
Block 27 thread 0 exiting critical section
Block 11 thread 0 entering critical section
[...]
Block 29 thread 0 entering critical section
Block 29 thread 0 exiting critical section
c=1024.0
```

OSS: l'uso di primitive native come `atomicAdd` è nettamente più efficiente, in quanto delega l'operazione in una singola transazione hardware. L'alternativa software (`atomicCAS` in un loop while) implementa uno spin-lock (attesa attiva) che causa grave congestione del bus di memoria a causa delle letture/scritture ripetute.

7. `cuda7_syncblocks.cu`

Implementazione di una barriera di sincronizzazione globale tra blocchi differenti per superare i limiti strutturali di `__syncthreads()`.

```
[mariaandrea.zoccoli@ui01 base]$ cat cuda7_syncblocks.out
[...]
Block 0 thread 0 entering sync-Blocks
[...]
Block:3-oldLock:3
Block:0-oldLock:4
[...]
Block 3 thread 0 exiting sync-Blocks
[...]
Finito, lock:0
```

OSS: Tutti thread "entrano", poi i blocchi comunicano il decremento di una variabile `lock`. Solo quando l'ultimo blocco decrementa il lock a 0, si sblocca l'attesa attiva (`while(lock){}`) e tutti iniziano ad uscire. È una pratica sconsigliata da NVIDIA in quanto spesso può generare deadlock se la GPU non ha abbastanza SM per caricare tutti i blocchi simultaneamente.

Script Opzionali

`cuda_vett+scalare.cu`

Lo script alloca un vettore di 100 elementi. Sulla CPU, lo inizializza con i valori del suo indice, poi copia questo vettore sulla memoria globale della GPU e lancia un kernel.

Ogni thread calcola il proprio ID univoco globale e lo usa come indice per accedere al vettore.

```
Primi 5 elementi dopo l'addizione dello scalare (5.0):
vett[0] = 5.0
vett[1] = 6.0
...
```

OSS: rappresenta l'esecuzione ideale su architettura SIMT, l'assoluta indipendenza dei dati garantisce il massimo dell'efficienza e del parallelismo computazionale.

`cuda_race.cu`

Simulazione forzata di una write-after-write race condition allocando una singola variabile in memoria shared a cui tutti i thread del warp tentano di assegnare il proprio ID contemporaneamente.

Esecuzione Race Condition (Write-after-Write in Shared Memory)

I thread da 0 a 31 hanno cercato di scrivere il loro ID.

Valore finale 'sopravvissuto' nella variabile condivisa: 0

Nota: Lanciando il programma più volte, questo numero potrebbe variare.

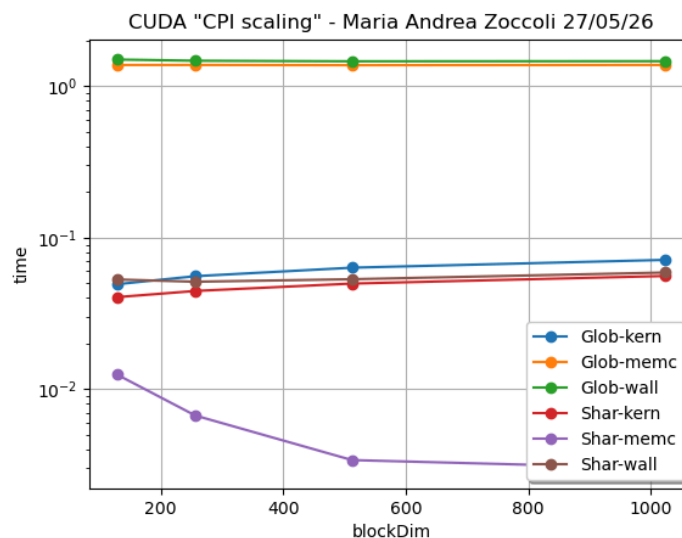
Trattandosi di un'architettura in cui tutti i thread di un warp eseguono la stessa istruzione nello stesso ciclo di clock, le scritture collidono. L'hardware risolve l'anomalia serializzando gli accessi, ma l'ordine di serializzazione non è deterministico, perciò il risultato finale dipende unicamente dall'ID del thread che riesce a completare l'operazione per ultimo sovrascrivendo gli altri.

Calcolo di Pi Greco

Il test valuta l'efficienza di due approcci di memoria (Global vs Shared) mantenendo invariato il carico di lavoro totale e scalando il parametro `blockDim`.

```
[mariaandrea.zoccoli@ui01 cpi]$ python3 cuda_cpi_scaling.py
```

	par	N	gridDim	blockDim	pi	err	wtime	mtime	ktime
0	CUDA	1000000000	7812500	128	3.052616	0.088976	1.5024	1.3820	0.0493
1	CUDA	1000000000	3906250	256	3.132408	0.009185	1.4751	1.3826	0.0557
2	CUDA	1000000000	1953125	512	3.139568	0.002025	1.4625	1.3777	0.0634
3	CUDA	999999488	976562	1024	3.143576	0.001984	1.4653	1.3824	0.0714



- **Collo di bottiglia del bus PCIe (mtime):**

Le linee del trasferimento dati (`Glob-memc` e `Shar-memc`) sono piatte, orizzontali e sovrapposte attorno al valore di ~1.38 secondi. Indipendentemente da come il carico venga ripartito tra i thread, spostare un miliardo di elementi tra RAM dell'Host e VRAM della GPU costituisce un collo di bottiglia strutturale hardware (I/O). L'algoritmo risulta pertanto fortemente *Memory-Bound*.

- **Shared vs Global:**

A differenza delle aspettative teoriche, secondo cui l'uso della shared dovrebbe abbattere i tempi di calcolo, nel grafico le linee `Glob-kern` e `Shar-kern` sono quasi sovrapposte, questo accade perché l'istruzione `res2[tid] =`

`p12 * 4 * h;` istruisce i thread a calcolare e scrivere il dato in celle di memoria contigue. L'hardware riconosce l'operazione e compatta le scritture (coalescing). Considerando che l'hardware è dotato di memorie ad altissima banda, eseguire scritture massive coalescenti in memoria globale risulta talmente rapido da annullare il vantaggio teorico della shared.

- **Effetto dello scaling e occupancy:**

L'aumento dei thread per blocco comporta oscillazioni del tempo minime, ciò dimostra il concetto di occupancy dei vari SM: alterare l'organizzazione dei blocchi modifica la densità di warp schedulati simultaneamente per mascherare le latenze. Le minime variazioni confermano che le performance dell'algoritmo dipendono dalle dinamiche di memoria e non dalla potenza computazionale.